

AWS re:Invent

NOV. 28 – DEC. 2, 2022 | LAS VEGAS, NV

ARC316-R1

Automating (near) zero-RPO disaster recovery across multiple AWS Regions

Zahi Ben Shabat

Sr. Prototype Architect
AWS

Alseny Diallo

Prototype Architect
AWS



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

**What do you want to know
about disaster recovery
with near zero-RPO?**

Automating failover across multiple regions

1. Introduction to resiliency
2. Why is multi-region resiliency hard?
3. Design an automated approach for failover
4. Chaos engineering and resiliency-testing best practices

Introduction to resiliency

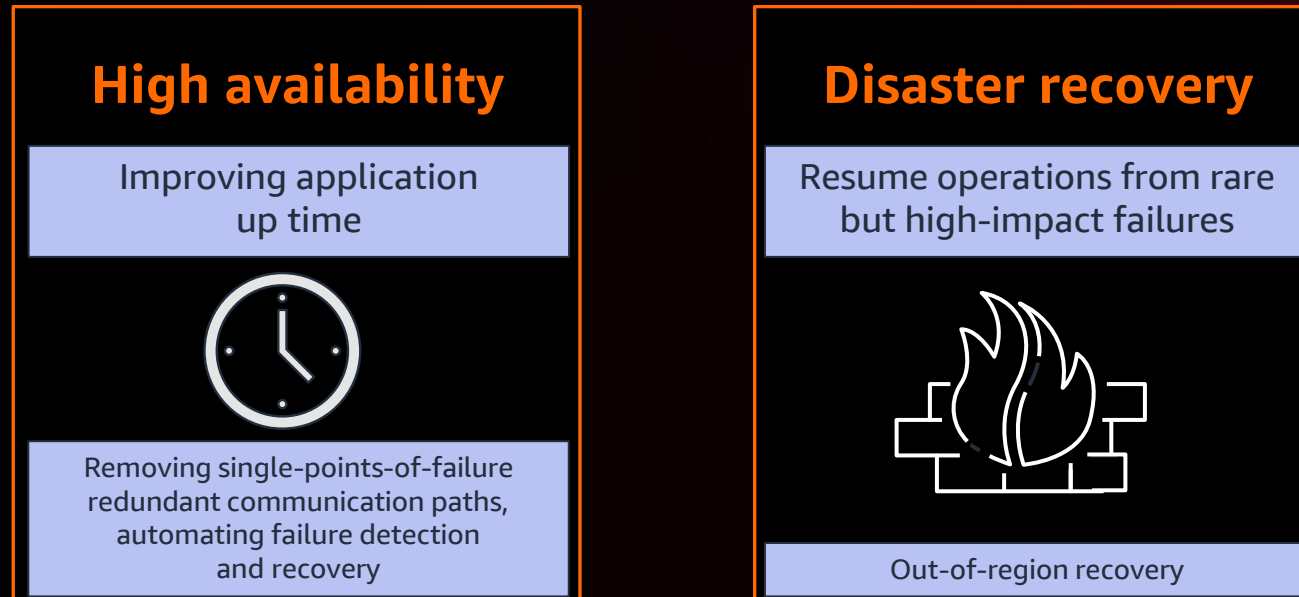


“Failure is success in progress.”

Albert Einstein

Disaster recovery vs. high availability

Resiliency is the ability to recover quickly from failures and resume operation



Critical - Resiliency affects the quality of service your users experience

Complex - End-to-end discipline that must accommodate different technologies

Cost driver - How many sites? How many data copies? What is the cost of the outage?

Cloud is different from traditional IT - Cloud offers better ways to provide continuity

RPO/RTO

How much data can you afford to recreate or lose?

**How quickly must you recover?
What is the cost of downtime?**



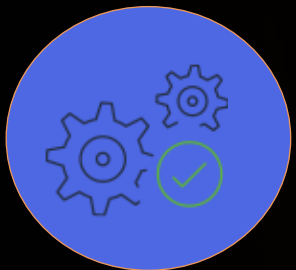
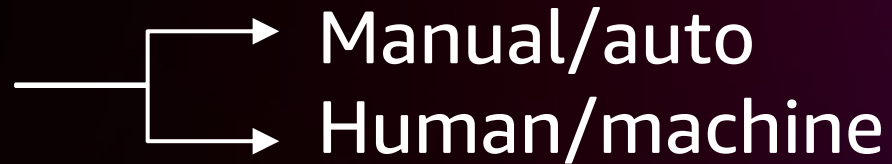
What do we need to achieve resiliency?



Monitoring



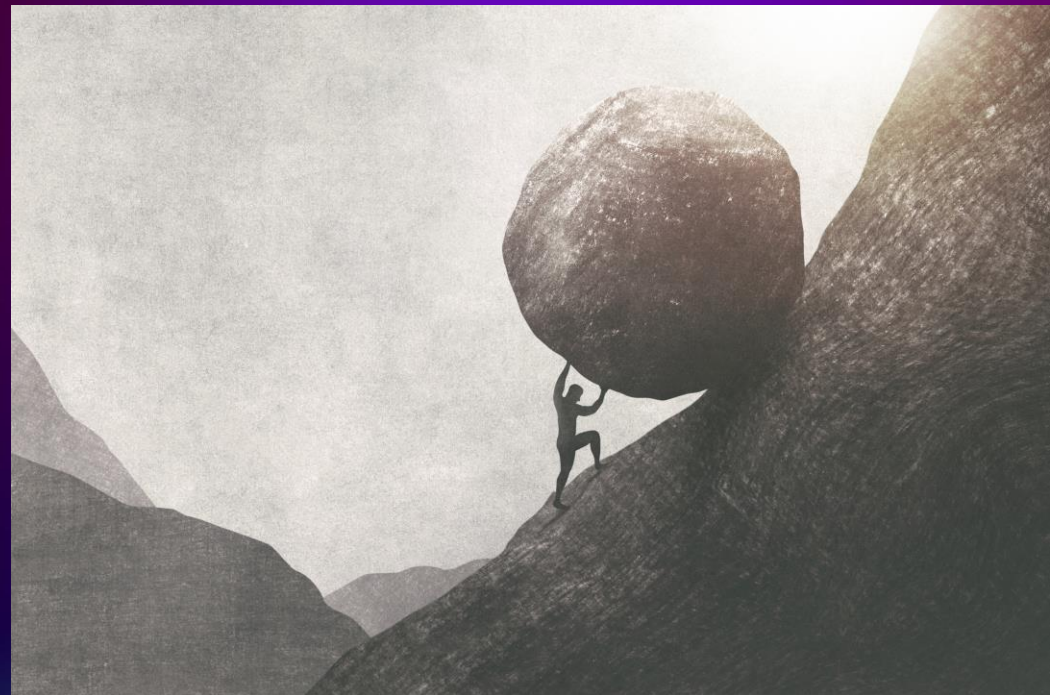
Decision-making



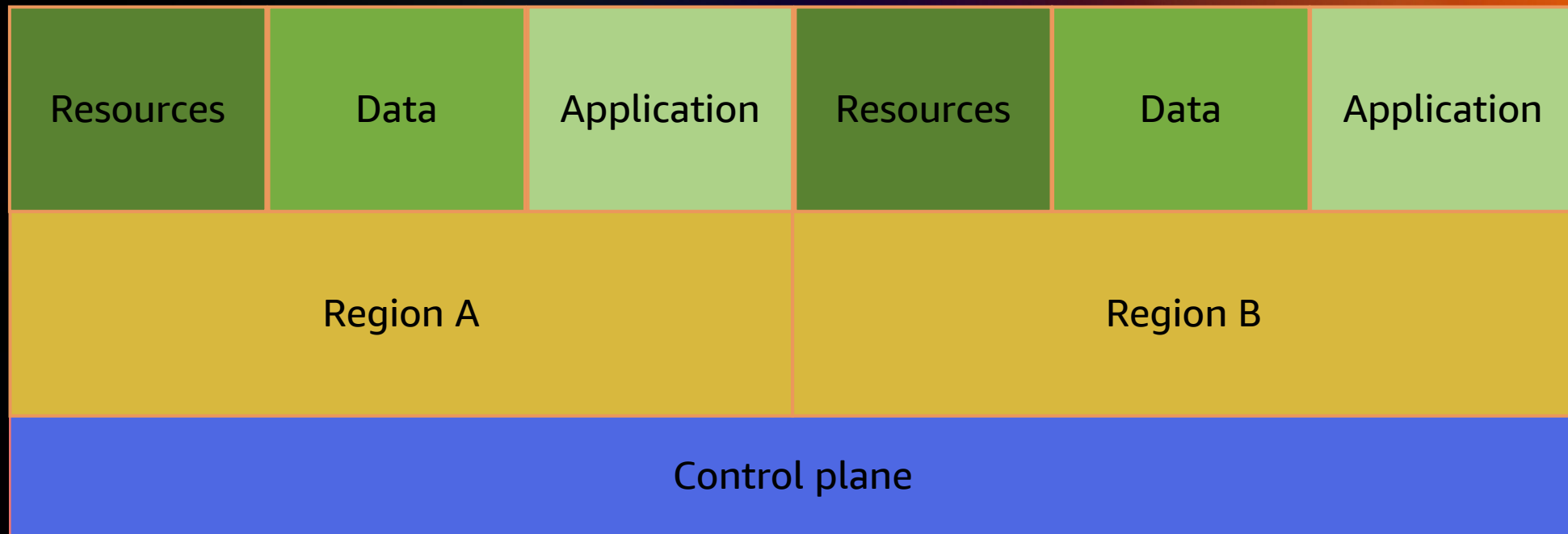
Orchestration



Why is multi-region resiliency hard?

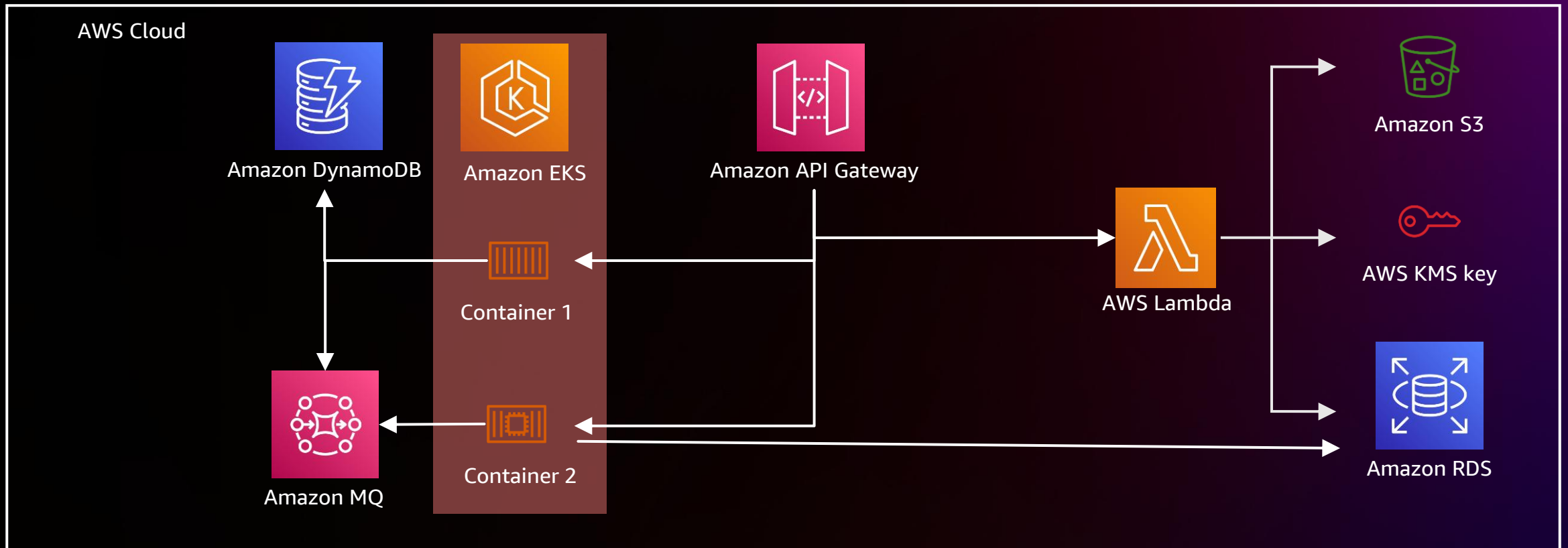


Multi-region resiliency challenges



Failover for complex applications

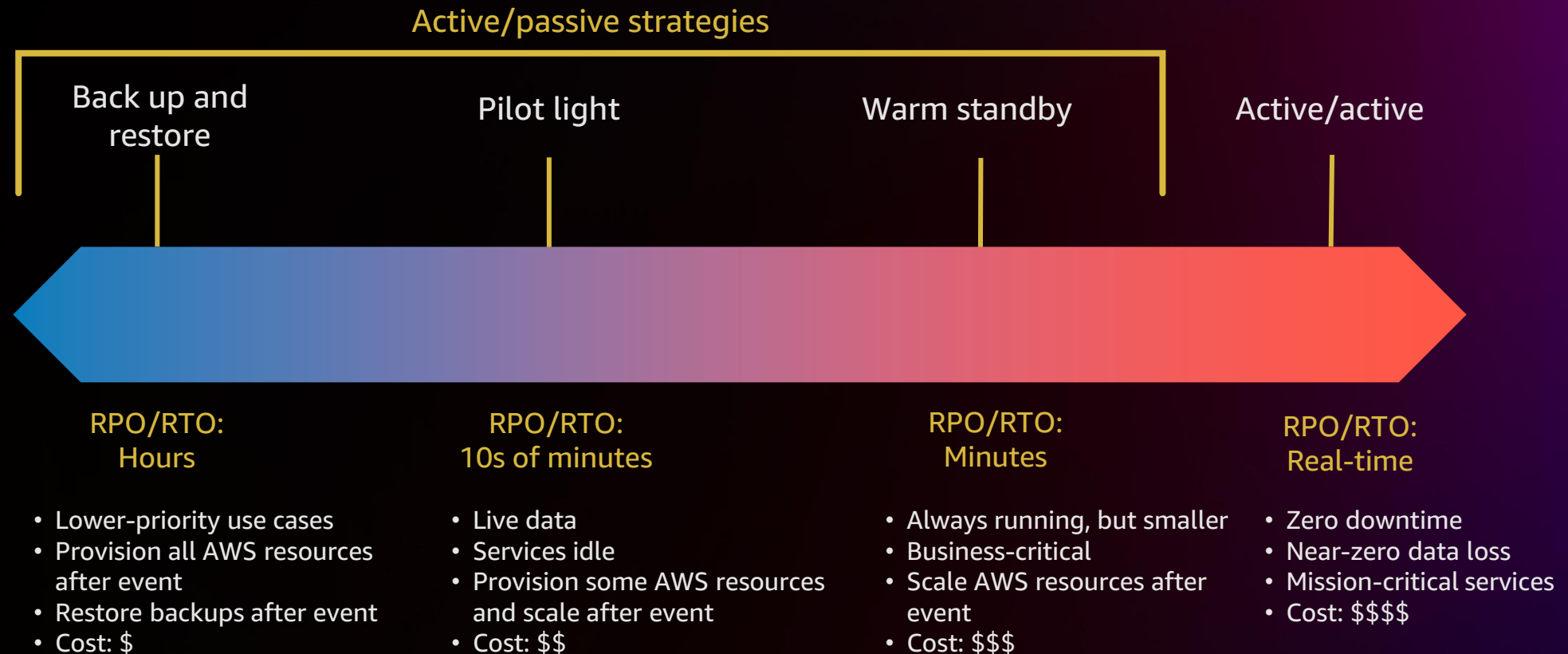
Complex application – Application that is composed of several components (micro-services) and uses various technologies



Design an automated approach for failover



Multi-region disaster recovery strategies



Manage data

Data replication

Use **AWS services** that support built-in data replication to easily distribute data across multiple regions



Amazon Aurora
global cluster



Amazon
DynamoDB



Amazon
MQ



Amazon
Kinesis



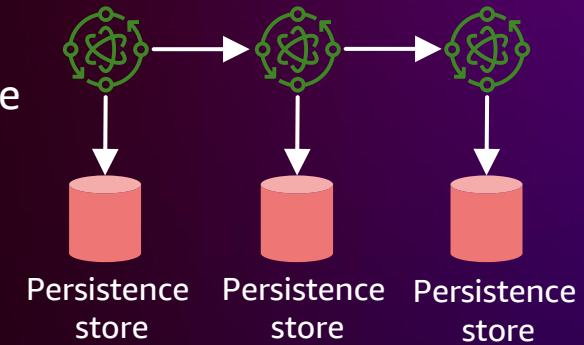
Amazon
S3

Transaction state

Maintain a transaction state over separate persistence stores for each processing step

Reduce **probability** of failure

Reduce **impact radius** of a failure



Is data healthy?

Handle in-flight
transactions

Data is properly
replicated

Otherwise, make sure data
can be recovered from origin

Manage resources

DEPENDS ON MULTI-REGION STRATEGY

	Resource	Implications
Back up and restore	Provision all AWS resources after event	Monitor the health and readiness of the new resources created
Pilot light	Provision some AWS resources and scale after event	Monitor the scale and sufficient capacity
Warm standby	Reserve smaller capacity in both regions	Monitor the scale and sufficient capacity
Active/active	Reserve capacity in both regions	Monitor the health of resources on both regions

Automate resource management

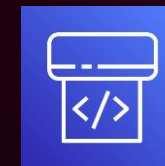
- ✓ Infrastructure as code
- ✓ Provision resources as required (reserve/on demand)
- ✓ Maintain resource state
- ✓ Configure resources



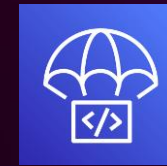
AWS CDK



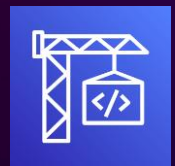
Terraform



AWS
CodePipeline



AWS
CodeDeploy



AWS
CodeBuild

Manage application

1 Reduce failure impact

- **Minimize** failure blast radius
- **Isolate** process-dependent resources
- **Reduce** probability of failure

2 Environment agnostic

- Application is designed to run in multiple regions
- Application fetches dependent configuration from its local environment
- Application is indifferent to the failure location and focused on exception type

3 Application exceptions

- Internal application **failure**
- Dependent downstream or upstream resource **failure**
- Persistent storage **failure**
- Executing environment **failure**
- Handle exceptions by:
 1. Attempt to recover (transient)
 2. Sleep and try again later (transient)
 3. Skip transaction (non-transient)
 4. Delegate responsibility (non-transient)

Utilize modern self-healing application environments

- Easily control application health
- Automatic recovery for current region
- Scalability

Implement **idempotency**

Implement application global state management



Amazon
EKS



Amazon
ECS



AWS Fargate

Automating Failover

DR automation runbook

Develop **Runbooks**

- **Automate** rotation
 - **Validates** system is operational
 - **Run data reconciliation** to mitigate data loss
-
- ✓ Carefully design beforehand
 - ✓ Faster execution
 - ✓ Region agnostic
 - ✓ Validated



Runbook

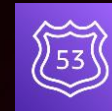


AWS
Systems Manager

DR resource readiness

Maintaining an Application **Global State**

- **AWS Route53 ARC** provides routing controls and access to change global flags across application
- **Readiness Check** verifies resources and application to ensure the data in both regions is consistent and in sync



Amazon Route 53



Readiness
checks



Routing controls



Application
recovery controller

Chaos engineering and resiliency testing best practices



How do you prove resiliency for an application?

Testing is key to prove that a competent system design can **withstand failures**.

This includes **predicting the failures** and **scenarios** a system may encounter, and then **confronting** the system with those same scenarios.

Initial assumption: **Application is in healthy state**



Build
hypothesis

1



Create
experiments

2



Execute
tests and validate

3

Simple failure scenario

1 Build **hypothesis**

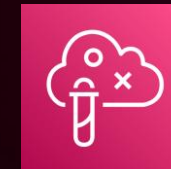


2 Create **experiments**

3 Execute **tests and validate**

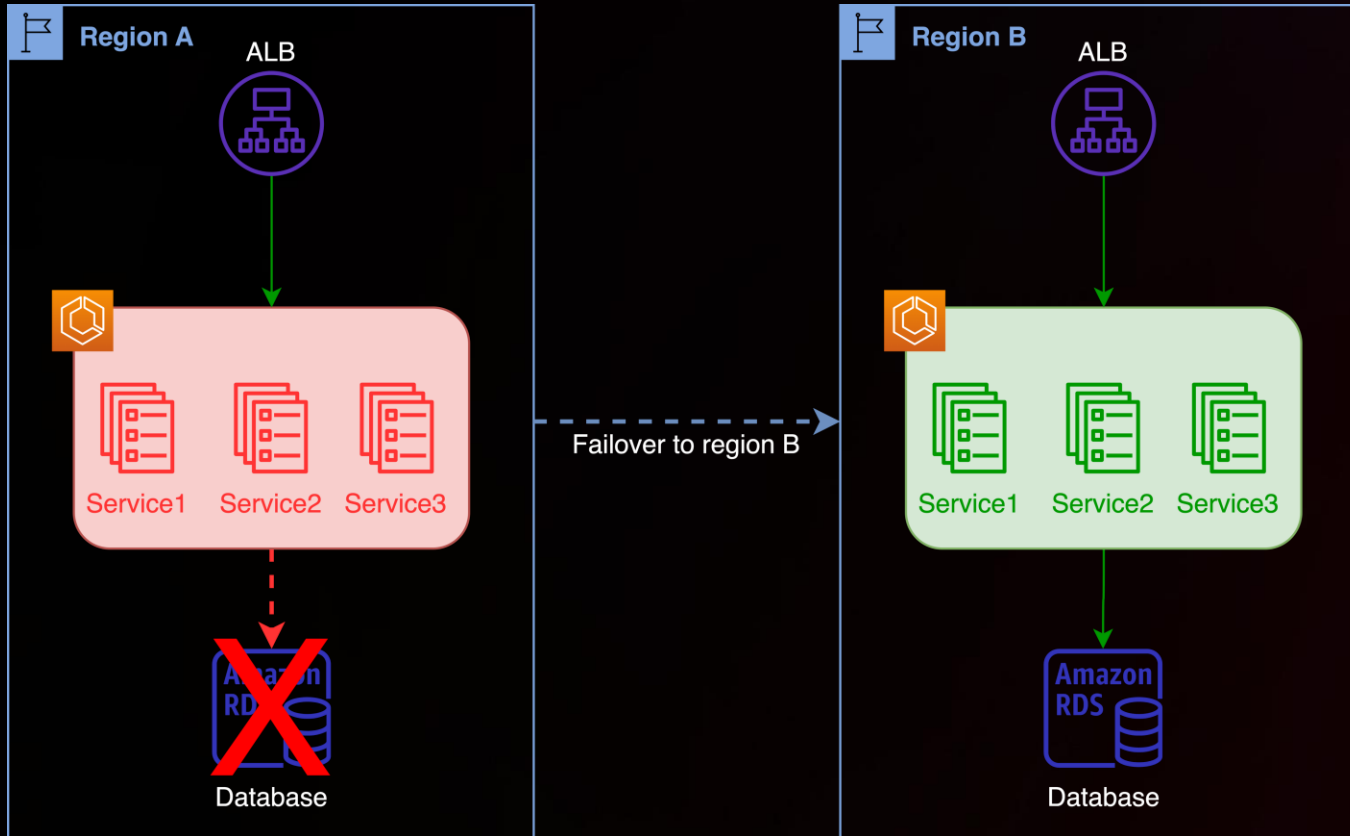


Simulate various **fault scenarios** and prove the application can **recover**



AWS Fault
Injection
Simulator

Application critical failure DR scenario



- 1 What happens during a **critical failure**
 - If **critical** failure occurs, application should **rotate** to **Region B**
- 2 Simulate **critical** application **failure**
 - Amazon ECS service failure
 - Database failure
- 3 Initiate **rotations** to **backup Region**
 - Execute **DR runbooks**
 - Execute **reconciliation** and **recovery**
 - Resume traffic in **backup Region**

Let's chalk



Thank you!



Linked in Zahi Ben Shabat



zshabat@amazon.com



Linked in Alseny Diallo



dialseny@amazon.com



Please complete the session survey in the **mobile app**

Want to learn more? aws-gfs-acceleration-amer@amazon.com



© 2022, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Additional resources

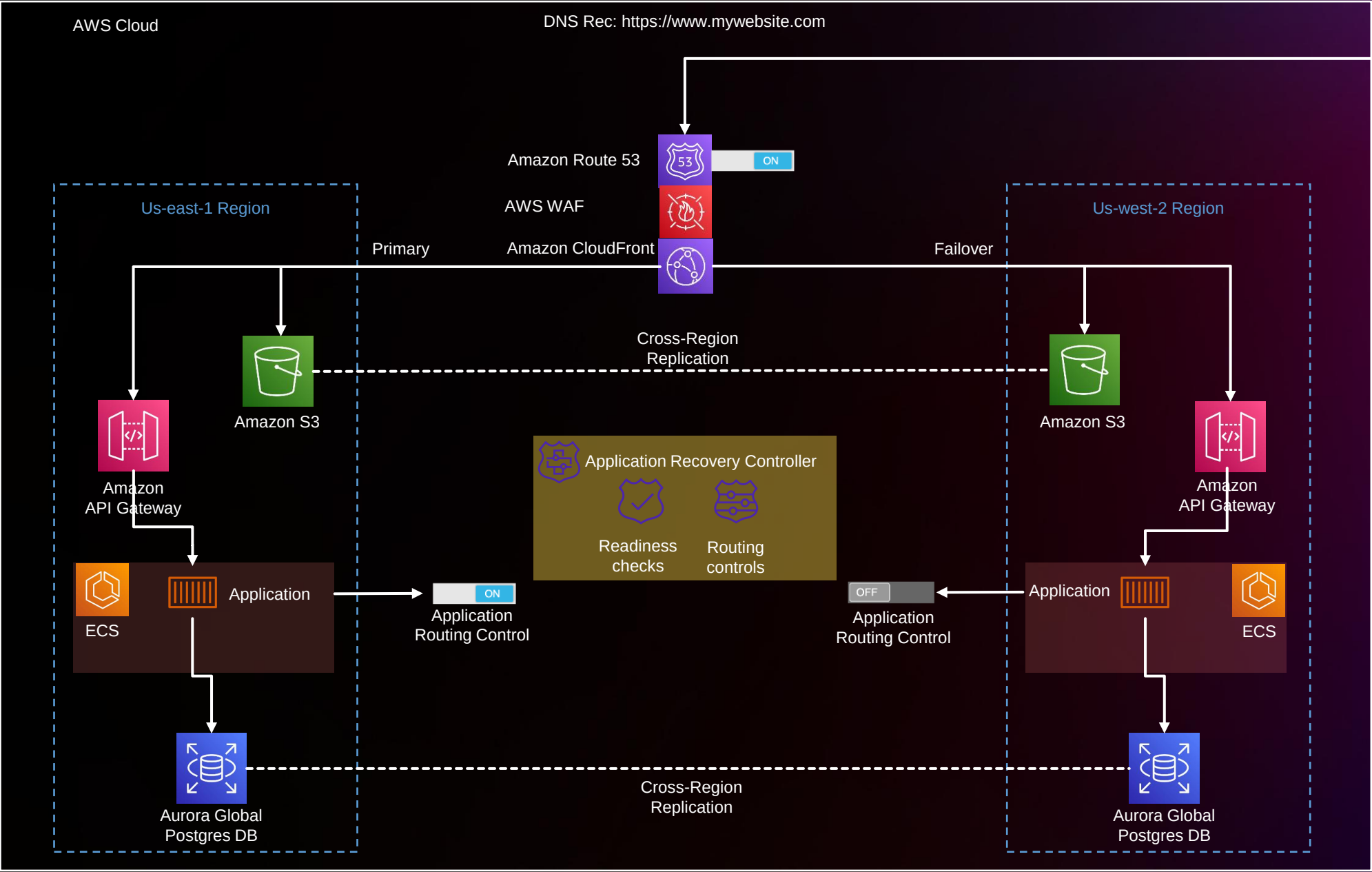


[Multi-region Resiliency
Reference Architecture \(GitHub\)](#)

Designing Highly Resilient Financial
Services Applications Whitepaper.
aws-gfs-acceleration-amer@amazon.com



References



Runbook steps

1. Determine current/active Region.
2. Turn off application routing control on secondary Region.
3. Turn Main DNS routing control off in active Region.
4. Wait for in-flight transaction in primary Region to finish processing.
5. Turn off application routing control on primary Region.
6. Wait for Amazon Aurora global database to complete replication to secondary Region
7. Initiate failover of the Aurora global database.
8. Wait for the failover of the Aurora global database to complete.
9. Update database secret.
10. Restart the ECS service on second Region.
11. Wait for application server to start up.
12. Turn on application routing controller on secondary.